1.Implementation of RPC

```
from xmlrpc.server import SimpleXMLRPCServer
import xmlrpc.client
import threading
import time

# ------------------ SERVER PART ------------------
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def start_server():
    server = SimpleXMLRPCServer(("localhost", 8000))
    print("Server started on port 8000...")

    server.register_function(add, "add")
    server.register_function(subtract, "subtract")

    server.serve_forever()

# Run server in separate thread
server_thread = threading.Thread(target=start_server)
server_thread.daemon = True
server_thread.start()

# Give server time to start
time.sleep(1)
```

```
# ------------------ CLIENT PART ------------------

client = xmlrpc.client.ServerProxy("http://localhost:8000/")


print("Addition:", client.add(10, 5))

print("Subtraction:", client.subtract(20, 8))
```

Output:

```
Server started on port 8000...
127.0.0.1 - - [26/Apr/2026 07:05:07] "POST / HTTP/1.1" 200 -
Addition: 15
127.0.0.1 - - [26/Apr/2026 07:05:07] "POST / HTTP/1.1" 200 -
Subtraction: 12

=== Code Execution Successful ===
```

2.Implementation of RPC by taking user data

```
from xmlrpc.server import SimpleXMLRPCServer
import xmlrpc.client
import threading
import time

# ------------------ SERVER PART ------------------
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def start_server():
    server = SimpleXMLRPCServer(("localhost", 8000))
    print("Server started on port 8000...")

    server.register_function(add, "add")
    server.register_function(subtract, "subtract")

    server.serve_forever()

# Run server in separate thread
server_thread = threading.Thread(target=start_server)
server_thread.daemon = True
server_thread.start()

# Give server time to start
```

```
time.sleep(1)


# ------------------ CLIENT PART ------------------

client = xmlrpc.client.ServerProxy("http://localhost:8000/")


# Get user input for addition

print("\n--- Addition ---")

num1_add = int(input("Enter the first number for addition: "))

num2_add = int(input("Enter the second number for addition: "))

print("Addition result:", client.add(num1_add, num2_add))


# Get user input for subtraction

print("\n--- Subtraction ---")

num1_sub = int(input("Enter the first number for subtraction: "))

num2_sub = int(input("Enter the second number for subtraction: "))

print("Subtraction result:", client.subtract(num1_sub, num2_sub))
```

Output:

```
Server started on port 8000...

--- Addition ---
Enter the first number for addition: 25
Enter the second number for addition: 67
127.0.0.1 - - [26/Apr/2026 07:08:17] "POST / HTTP/1.1" 200 -
Addition result: 92

--- Subtraction ---
Enter the first number for subtraction: 78
Enter the second number for subtraction: 23
127.0.0.1 - - [26/Apr/2026 07:08:25] "POST / HTTP/1.1" 200 -
Subtraction result: 55

=== Code Execution Successful ===
```

3.Implementation of RMI

```
import socket
import threading
import json
# ---------- SERVER PART ----------
def handle_client(conn, addr):
    print(f"Connected by {addr}")
    data = conn.recv(1024).decode()
    if not data:
        return
    request = json.loads(data)
    method = request.get("method")
    params = request.get("params", [])
    # Remote methods
    def add(a, b):
        return a + b
    def subtract(a, b):
        return a - b
    # Method dispatcher
    if method == "add":
        result = add(*params)
    elif method == "subtract":
        result = subtract(*params)
    else:
        result = "Method not found"
    response = json.dumps({"result": result})
    conn.send(response.encode())
    conn.close()
```

```
def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(("localhost", 5000))
    server.listen(5)
    print("RMI Server running on port 5000...")
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr)).start()
# ---------- CLIENT PART ----------
def call_remote_method(method, params):
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(("localhost", 5000))
    request = json.dumps({
        "method": method,
        "params": params
    })
    client.send(request.encode())
    response = client.recv(1024).decode()
    result = json.loads(response)
    client.close()
    return result["result"]
# ---------- MAIN ----------
if __name__ == "__main__":
    import time
    # Start server in background
    threading.Thread(target=start_server, daemon=True).start()
    time.sleep(1)  # Give server time to start
```

```
# Client calls
    print("Calling remote add(5, 3)...")
    print("Result:", call_remote_method("add", [5, 3]))
    print("Calling remote subtract(10, 4)...")
    print("Result:", call_remote_method("subtract", [10, 4]))
```

Output:

```
RMI Server running on port 5000...

--- RMI Addition ---
Enter the first number for RMI addition: 49
Enter the second number for RMI addition: 74
Connected by ('127.0.0.1', 43978)
Result of RMI addition: 123

--- RMI Subtraction ---
Enter the first number for RMI subtraction: 29
Enter the second number for RMI subtraction: 43
Connected by ('127.0.0.1', 42990)
Result of RMI subtraction: -14

=== Code Execution Successful ===
```

4.Implementation of RMI by taking user data

```
import socket
import threading
import json
# ---------- SERVER PART ----------
def handle_client(conn, addr):
    print(f"Connected by {addr}")
    data = conn.recv(1024).decode()
    if not data:
        return
    request = json.loads(data)
    method = request.get("method")
    params = request.get("params", [])
    # Remote methods
    def add(a, b):
        return a + b
    def subtract(a, b):
        return a - b
    # Method dispatcher
    if method == "add":
        result = add(*params)
    elif method == "subtract":
        result = subtract(*params)
    else:
        result = "Method not found"
    response = json.dumps({"result": result})
    conn.send(response.encode())
    conn.close()
```

```
def start_server():
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind(("localhost", 5000))
    server.listen(5)
    print("RMI Server running on port 5000...")
    while True:
        conn, addr = server.accept()
        threading.Thread(target=handle_client, args=(conn, addr)).start()
# ---------- CLIENT PART ----------
def call_remote_method(method, params):
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect(("localhost", 5000))
    request = json.dumps({
        "method": method,
        "params": params
    })
    client.send(request.encode())
    response = client.recv(1024).decode()
    result = json.loads(response)
    client.close()
    return result["result"]
# ---------- MAIN ----------
if __name__ == "__main__":
    import time
    # Start server in background
    threading.Thread(target=start_server, daemon=True).start()
    time.sleep(1)  # Give server time to start
```

```
# Client calls with user input
    print("\n--- RMI Addition ---")
    num1_add = int(input("Enter the first number for RMI addition: "))
    num2_add = int(input("Enter the second number for RMI addition: "))
    print("Result of RMI addition:", call_remote_method("add", [num1_add, num2_add]))
    print("\n--- RMI Subtraction ---")
    num1_sub = int(input("Enter the first number for RMI subtraction: "))
    num2_sub = int(input("Enter the second number for RMI subtraction: "))
    print("Result of RMI subtraction:", call_remote_method("subtract", [num1_sub, num2_sub]))
```

Output:

```
RMI Server running on port 5000...

--- RMI Addition ---
Enter the first number for RMI addition: 56
Enter the second number for RMI addition: 89
Connected by ('127.0.0.1', 50512)
Result of RMI addition: 145

--- RMI Subtraction ---
Enter the first number for RMI subtraction: 90
Enter the second number for RMI subtraction: 76
Connected by ('127.0.0.1', 50872)
Result of RMI subtraction: 14

=== Code Execution Successful ===
```

5.Simulation of Lamport Logical Clocks

```
p1 = 0

p2 = 0

p3 = 0


p1 += 1

p1 += 1

print("P1 sends message to P2 at time", p1)


p2 = max(p2, p1) + 1

print("P2 receives message at time", p2)


p2 += 1

print("P2 sends message to P3 at time", p2)


p3 = max(p3, p2) + 1

print("P3 receives message at time", p3)
```

Output:

```
P1 sends message to P2 at time 2
P2 receives message at time 3
P2 sends message to P3 at time 4
P3 receives message at time 5

=== Code Execution Successful ===
```

6.Simulation of Mutual Exclusion

```
N = 3

for i in range(N):
    print("Process", i, "sending REQUEST to other processes")

    replies = int(input("Enter number of REPLY received: "))

    if replies == N-1:
        print("Process", i, "ENTERING Critical Section")
        print("Process", i, "EXITING Critical Section\n")

    else:
        print("Process", i, "WAITING (Some process did not reply)\n")
```

Output:

```
Process 0 sending REQUEST to other processes
Enter number of REPLY received: 1
Process 0 WAITING (Some process did not reply)

Process 1 sending REQUEST to other processes
Enter number of REPLY received: 2
Process 1 ENTERING Critical Section
Process 1 EXITING Critical Section

Process 2 sending REQUEST to other processes
Enter number of REPLY received: 3
Process 2 WAITING (Some process did not reply)


=== Code Execution Successful ===
```

7.Deadlock Detection

```
import collections

# A simplified representation of a distributed wait-for graph

# Each key (process_id) waits for the process_id in its value list.

# For example: wait_for_graph = {"P1": ["P2"], "P2": ["P3"], "P3": ["P1"]} would represent a deadlock.

def detect_deadlock(wait_for_graph):

    """

    Detects deadlocks in a wait-for graph using Depth First Search (DFS).

    A cycle in the wait-for graph indicates a deadlock.

    Returns True if a deadlock is detected, along with a list of processes

    involved in the detected cycle, otherwise False and an empty list.

    """

    visited = set()

    recursion_stack = collections.deque()  # To store the current path in DFS

    def dfs(process):

        visited.add(process)

        recursion_stack.append(process)

        for neighbor in wait_for_graph.get(process, []):

            if neighbor not in visited:

                cycle = dfs(neighbor)

                if cycle:

                    return cycle

            elif neighbor in recursion_stack:

                # Cycle detected! Reconstruct the cycle.

                # The cycle starts from 'neighbor' and goes through processes

                # currently in the recursion_stack until 'process'.

                cycle_start_index = list(recursion_stack).index(neighbor)

                return list(recursion_stack)[cycle_start_index:] + [neighbor]
```

```
        recursion_stack.pop()  # Backtrack
        return None
    for process in wait_for_graph:
        if process not in visited:
            cycle = dfs(process)
            if cycle:
                return True, cycle
    return False, []
# --- Example Usage ---
print("--- Distributed Deadlock Detection Simulation ---")
# Scenario 1: No Deadlock
print("\nScenario 1: No Deadlock")
wait_for_graph_no_deadlock = {
    "P1": ["P2"],
    "P2": ["P3"],
    "P3": []
}
print("Wait-for graph:", wait_for_graph_no_deadlock)
deadlock_detected, cycle = detect_deadlock(wait_for_graph_no_deadlock)
if deadlock_detected:
    print("Deadlock detected! Processes involved in cycle:", cycle)
else:
    print("No deadlock detected.")
# Scenario 2: Deadlock Detected (P1 -> P2 -> P3 -> P1)
print("\nScenario 2: Deadlock Detected (P1 -> P2 -> P3 -> P1)")
```

```
wait_for_graph_deadlock = {
    "P1": ["P2"],
    "P2": ["P3"],
    "P3": ["P1"]
}
print("Wait-for graph:", wait_for_graph_deadlock)
deadlock_detected, cycle = detect_deadlock(wait_for_graph_deadlock)
if deadlock_detected:
    print("Deadlock detected! Processes involved in cycle:", cycle)
else:
    print("No deadlock detected.")
# Scenario 3: Another Deadlock (more complex: P_A -> P_B -> P_C -> P_A)
print("\nScenario 3: Another Deadlock Detected (P_A -> P_B -> P_C -> P_A)")
wait_for_graph_deadlock_2 = {
    "P_A": ["P_B"],
    "P_B": ["P_C"],
    "P_C": ["P_A", "P_D"], # P_C waits for P_A, creating a cycle
    "P_D": []
}
print("Wait-for graph:", wait_for_graph_deadlock_2)
deadlock_detected, cycle = detect_deadlock(wait_for_graph_deadlock_2)
if deadlock_detected:
    print("Deadlock detected! Processes involved in cycle:", cycle)
else:
    print("No deadlock detected.")
# Scenario 4: No Deadlock with multiple paths
print("\nScenario 4: No Deadlock with multiple paths")
```

```
wait_for_graph_no_deadlock_2 = {
    "P_X": ["P_Y", "P_Z"],
    "P_Y": [],
    "P_Z": ["P_W"],
    "P_W": []
}
print("Wait-for graph:", wait_for_graph_no_deadlock_2)
deadlock_detected, cycle = detect_deadlock(wait_for_graph_no_deadlock_2)
if deadlock_detected:
    print("Deadlock detected! Processes involved in cycle:", cycle)
else:
    print("No deadlock detected.")
```

Output:

```
--- Distributed Deadlock Detection Simulation ---

Scenario 1: No Deadlock
Wait-for graph: {'P1': ['P2'], 'P2': ['P3'], 'P3': []}
No deadlock detected.

Scenario 2: Deadlock Detected (P1 -> P2 -> P3 -> P1)
Wait-for graph: {'P1': ['P2'], 'P2': ['P3'], 'P3': ['P1']}
Deadlock detected! Processes involved in cycle: ['P1', 'P2', 'P3', 'P1']

Scenario 3: Another Deadlock Detected (P_A -> P_B -> P_C -> P_A)
Wait-for graph: {'P_A': ['P_B'], 'P_B': ['P_C'], 'P_C': ['P_A', 'P_D'], 'P_D':
    []}
Deadlock detected! Processes involved in cycle: ['P_A', 'P_B', 'P_C', 'P_A']

Scenario 4: No Deadlock with multiple paths
Wait-for graph: {'P_X': ['P_Y', 'P_Z'], 'P_Y': [], 'P_Z': ['P_W'], 'P_W': []}
No deadlock detected.

=== Code Execution Successful ===
```

8.Manual Testing

```
keywords = {
  "war": "Fake News",
  "attack": "Fake News",
  "election": "Real News",
  "government": "Real News"
}
def predict(text):
  text = text.lower().strip()
  for key in keywords:
    if key in text:
      return keywords[key]
  return "Uncertain Prediction"
while True:
  text = input("Enter a news article (or 'exit' to quit): ")
  if text.lower() == "exit":
    break
  print("Prediction model:", predict(text))
```

OUTPUT :

Enter a news article (or 'exit' to quit): The government announced reforms

Prediction model: Real News

Enter a news article (or 'exit' to quit): War breaks out in the region

Prediction model: Fake News

Enter a news article (or 'exit' to quit): Sports update

Prediction model: Uncertain Prediction